

graph simulate

Coverage-aware change simulation you are allowed to distrust.

Track	Track 2 — Build with Graph Intelligence
Command added	entire graph simulate
Entire mirror	entire://aws-ap-south-1.entire.io/gh/arenforge/entire-graph-deathcode
Mirror ID	01M1TM59AMZQ38B7TCC4BFP25A
GitHub fork	https://github.com/arenforge/entire-graph-deathcode
Branch	buildathon/main
Final commit	492c3a84fa17330ac8099ae3167ad0615863be9c
Databricks	Not applicable — this team did not opt in to that award category

One-sentence summary

`entire graph simulate` takes the blast radius `impact` already computes and answers the question it leaves open — *of everything my change can reach, which parts does a test actually guard?* — labelling every claim with the strength of the evidence behind it, and handing back a command for every claim it cannot prove.

Problem, intended user, and why it matters

The user is a developer about to change a function, or a reviewer reading a change an agent wrote. Two questions decide whether it is safe to merge: **what else depends on this**, and **would anything catch me if I got it wrong?**

This repository already answered the first. `entire graph impact` reports callers, callees, type consumers and data flows for any symbol. Nothing answered the second — and the pieces were sitting right next to each other:

- `internal/sem/search_covertest.go` implements a tiered covering-test resolver, but it is wired only into `search`, returns exactly **one** test, and only for the **top hit**.
- `internal/cli/impact.go` has **zero** mentions of `TESTS`. The blast-radius command has no notion of coverage at all.

So the tool could say "here is one test for one symbol" and "here is everything that depends on one symbol", and nothing composed them. A blast radius of twelve symbols arrived with no indication of which six were unguarded. That composition is this command.

Why Entire Graph is essential

The product cannot exist without the graph. Mapping a symbol to its transitive callers, then those callers to the test functions that reach them, is a call-graph query, not a text query. Entire Graph supplies it locally, with no model calls, and returns `file:line` for every node — which is what makes the verdict *auditable* rather than advisory.

The graph also chose the feature. We did not guess at a gap; we found it with `entire graph search` and confirmed it by opening every cited line.

Architecture

```
--symbol NAME | <file>:<line>
|
v shared focus resolver (ambiguity answered exactly as impact answers it)
parseImpactFlags                                internal/cli/simulate.go:348
|
v ONE snapshot load, no network
sem.LoadOrBuildProviderSnapshot                internal/cli/simulate.go:305
|
+--> blast radius, REUSED not recomputed
|   buildImpactResponseFromReader  internal/cli/impact.go:330
|
+--> coverage index, built once per run (not once per symbol)
|   newSimulateCoverageIndex      internal/cli/simulate.go:584
|
v correlate, tier by resolution strength, classify
buildSimulateResponse                    internal/cli/simulate.go:360
|
v
AFFECTED / TESTS / BREAKING / UNCOVERED / VERIFY / RISK  (text | json)
writeSimulateText                        internal/cli/simulate.go:939
```

File	Role
internal/cli/simulate.go	The whole command
internal/cli/simulate_test.go	17 tests, including the dynamic-dispatch fixture
internal/cli/root.go:104	case "simulate":
internal/cli/help.go:207	Help entry
internal/cli/completeness.go	Reused for partial-analysis reporting, <i>not modified</i>
docs/buildathon/evidence/	Captured runs

TWO LOAD-BEARING REUSE DECISIONS

The blast radius is `impact` 's, unchanged. If `simulate` recomputed it, the two commands could disagree about what "affected" means, and a developer who ran both would have no way to know which to believe.

Partial-analysis reporting is `completeness.go` 's, unchanged. `impact` already builds a query-scoped completeness view and renders it. Rather than write a second one, `simulate` now carries the same scope and calls the same writer.

What the command reports

Six blocks — `AFFECTED` , `TESTS` , `BREAKING` , `UNCOVERED` , `VERIFY` , `RISK` — every line carrying `file:line` and the strength of the evidence behind it. Text and JSON.

Coverage is reported in **three states, never as a certainty**:

State	Meaning	Tier printed
confirmed	The provider traced the call — resolution <code>exact</code> , <code>import_resolved</code> or <code>package</code>	<code>edge</code>
heuristic	The edge exists but was drawn by matching a name, a pattern, or an inferred receiver type. Real evidence; <i>not</i> proof of execution	<code>edge-heuristic</code> , <code>mirror</code>
unresolved	No test was <i>resolved</i> . Not the same claim as "no test exists"	—

Every claim that is not confirmed appears under `VERIFY` with the command that settles it. `ANALYSIS MAY BE PARTIAL` fires when any input was incomplete and names each reason; JSON exposes `analysis_partial` plus `partial_reasons` , so a merge gate can act without parsing text.

`RISK` is a four-level verdict (`NONE` / `LOW` / `MEDIUM` / `HIGH`) **recomputable by hand from the two printed counts**. No scores, no confidence percentages.

Noon curveball: the graph is evidence, not an oracle

The assumption it invalidated

`simulate` treated the graph as a **closed world**: an absent relation was read as a negative fact about the code, and any present relation as proof of execution. Both halves lived in one function — `len(entry.Tests) == 0` became the affirmative claim "has no covering test" and drove the `RISK` verdict, while any inbound edge from a test file was stamped tier `edge`, documented as *"the graph proved a test actually runs this code"*.

Dynamic dispatch, reflection and generated code produce exactly the two states that assumption has no room for: a real test whose path the resolver cannot follow, and an edge drawn only because one name happened to match.

We ran the graph on our own code before editing any of it

`impact` and `simulate` on all eight of our graph-evidence consumers, saved to `evidence/04-before-curveball.txt` (689 lines), captured at `df3bf1b` with nothing under `internal/` modified. It found four defects — each **measured, not asserted**:

1. All **9 TESTS** edges in this repository are resolved `name_only` (8) or `package` (1). **Zero are exact**. We were labelling those "proven execution".
2. `impact` printed `Completeness: degraded for Go (...)` on all 8 runs; `simulate` printed it **0 times** on the identical snapshot. The diagnostics were plumbed into our response object and never read — so a snapshot the provider itself flags as degraded produced a bare `RISK HIGH`.
3. `collectSimulateAffected` read `section.Entries` and never `section.Total`, so a capped blast radius silently shrank the risk denominator while the header stated it as fact.
4. `impact`'s own `doc-mention`, `name_only` label was dropped, promoting a lexical mention in a prose file to a fully affected symbol.

The fix

- **Evidence is tiered by the provider's own Resolution**, not by relation type. The partition is read off the provider's `Reason` strings and documented at `simulateConfirmedResolution`. An *unknown* resolution degrades toward verification, never toward proof.
- **Three coverage states** in text and JSON. `unresolved` is never called "no test exists".
- **Partial analysis is rendered**, reusing `writeScopedCompletenessBlock` and `completenessScopeOrAll` unchanged, plus an `ANALYSIS MAY BE PARTIAL` block naming every reason and an `analysis_partial` boolean for gates.
- **Truncation surfaced**: `affected_omitted_total`, and the header states the totals are a lower bound.
- **A VERIFY section**: the one test to run for a heuristic claim, `neighbors --direction in` for an unresolved one. The `Go -run` form is emitted only for Go test files — for other languages the runner is *not guessed*.
- **format_version 1 → 2**. `evidence` gained a third value, which is a compatibility break for a consumer switching exhaustively on the v1 tiers.

REQUIREMENT 4 — WHAT DID NOT CHANGE, PINNED BY TEST

For a symbol whose test edge is resolved: tier still `edge`, verdict `confirmed`, identical covered/uncovered classification, `RISK` thresholds untouched. In this repository that is **1292 of 1405** covered symbols. The defect affected the other **113**. The live run on `buildImpactResponseFromReader` is unchanged at 12 affected / 6 covered / 6 uncovered / 1 cross-module / `RISK HIGH`.

The fixture: proof the tool is honest

A repository where `Gateway` has **two** implementations, so no unique implementing method exists and the interface call cannot be resolved; `Charge` is not globally unique either, so the name-matching fallback cannot fire. The hop is *genuinely* unresolvable, not unresolved by luck.

```
func Checkout(g Gateway, cents int) error { return g.Charge(cents) } // <- the blind spot

func TestCheckoutWithStripe(t *testing.T) {
    Checkout(Stripe{}, 100) // really does execute chargeViaStripe
}
```

Ground truth, measured by Go's own coverage tool:

```
$ go test ./... -run TestCheckoutWithStripe -coverprofile=cov.out
$ go tool cover -func=cov.out

fixture/payments.go:15: chargeViaStripe 100.0%
```

`chargeViaStripe` is **100% covered**, and the graph cannot see it. The old output said "*no covering test*" — which was **false**. The new output:

```
UNCOVERED 2 affected symbols have no test the graph could resolve
           (that is an absence of resolved evidence, not proof no test exists)

VERIFY    2 claims are not confirmed structural evidence
chargeViaStripe payments.go:15 no test the graph could resolve
$ entire graph neighbors --repo . --symbol chargeViaStripe --relation CALLS --direction in

ANALYSIS MAY BE PARTIAL - this verdict rests on incomplete evidence:
- 2 symbols have no test the graph could resolve, which is not proof no test exists
```

Both states — the blind spot and the fully resolved case in the same repository — are asserted in `TestSimulateOnUnresolvableDynamicDispatchFixture`.

Setup, run and test

```
git clone entire://aws-ap-south-1.entire.io/gh/arenforge/entire-graph-deathcode
cd entire-graph-deathcode && git checkout buildathon/main
go build -o entire-graph ./cmd/entire-graph

# --head AND --cache-dir are both required for the index cache to hit:
# 22.0s on the first run, 0.637s on every run after.
./entire-graph simulate --repo . --symbol buildImpactResponseFromReader \
  --head --cache-dir .cache/graph

# machine output, for the merge-gate story
./entire-graph simulate --repo . --symbol buildImpactResponseFromReader \
  --head --cache-dir .cache/graph --format json | jq '{analysis_partial, partial_reasons, verify}'

# tests (the env vars keep fixtures from inheriting global git config)
GIT_CONFIG_GLOBAL=/dev/null GIT_CONFIG_SYSTEM=/dev/null go test -timeout 40m ./...
```

Toolchain: Go 1.26, Git ≥ 2.36, Entire CLI ≥ 0.10.0, Entire Graph v0.4.0.

Verification

Check	Result
go build ./...	clean
gofmt -l ./cmd ./internal	clean
go vet ./internal/cli/	clean
go test -timeout 40m ./...	ok, all 8 packages, exit 0
Tests in simulate_test.go	6 → 17

The full suite passing is new — it had never been run on this branch. Doing so surfaced a **real pre-existing bug**: `simulate` was in `root.go` 's dispatch switch and `help.go` 's registry but missing from the `dispatchCommands` mirror list in `help_test.go` , so `TestRegistryMatchesDispatch` had been failing since the command was first added. Fixed, one line.

Evidence artifacts

#	Requirement	Artifact
1	Graph search / definition lookup	01-graph-search.txt
2	Impact analysis before a high-risk change	02-impact-pre-change.txt
3	The command running on real code	03-simulate-run.txt
4	Graph analysis before the curveball edit	04-before-curveball.txt (689 lines)
5	Final semantic diff of the submission	05-final-semantic-diff.txt
6	The incomplete-analysis fixture + ground truth	06-curveball-fixture.txt

All under `docs/buildathon/evidence/` . Full write-up: `BUILDATHON.md` . Plain-language brief: `docs/buildathon/TEAM-BRIEF.md` .

Checkpoints and what each proves

Milestone	Commit	What it proves
Initial understanding and architecture	651d2a0	The gap was found with the graph and confirmed against source; rejected alternatives recorded with reasons
Cross-module blast radius	b9dbba6	Last stable state before the curveball, with open risks listed
Found by running it	dca1066	"tests": [] vs null are different claims; caught by a teammate's real run
Plain-language handover	df3bf1b	Three people can present work they did not personally write
Curveball response	d4ea3f5	Graph analysis captured <i>before</i> the edit; closed-world assumption removed; fixture with measured ground truth
Final evidence + demo fix	881db89 , 492c3a8	Semantic diff captured; cache-flag defect found by running it

Design decisions we deliberately rejected

Rejected	Why
A score like "Risk 8.9/10" or "Confidence 94%"	No calibrated basis for a decimal. This repo's own README spends ~40 lines retracting over-precise benchmark figures.
An LLM in the decision path	The provider "makes no network requests, model calls, or API-key lookups". Adding one would contradict the product we are extending. All correlation is deterministic.
Callees in the affected set	Changing a function does not break the functions it calls.
Co-changes and siblings	Correlations, not dependencies. A coverage gap reported against something a change cannot break is a false alarm — and one false alarm ends a reviewer's trust in the whole report.
Leading with blast radius / affected modules	impact already ships those. A reskin is not a new capability.
Deployment checklists, suggested reviewers	Neither is derivable from a call graph. Listed as next steps rather than invented.
Mirror-tier credit from filename adjacency alone	The test must also <i>name</i> the anchor, or every test in foo_test.go would "cover" every symbol in foo.go — exactly the unearned coverage claim this command exists to expose.

Known limitations

1. **"Confirmed"** means a test *reaches* the symbol, not that it *asserts the behaviour you are changing*. That is the honest ceiling of the whole approach.
2. TESTS is a heuristic relation type here, and all 9 in this repo are resolved by name. The command prints this itself, on every run.
3. **"Unresolved"** cannot distinguish "no test" from "a test we cannot see." We report the ambiguity rather than resolve it; that is what VERIFY is for. A future version could run the cited tests under coverage and close the loop.
4. The confirmed/heuristic partition is a judgement, though every value in it is read off the provider's own reason strings. package counts as confirmed ("*direct call expression resolved to same-package symbol*"); type_inferred does not. Pinned by TestSimulateConfirmedResolutionPartition — that is where to argue about it.

5. Verified on this repository plus a Go fixture. Stem matching is unit-tested against TS/Python/Ruby naming and the resolution partition is language-independent, but neither has run on a real project in those languages.
6. The `go test -run` verification command is emitted for Go only; for other languages we print "open this file" rather than guess a test runner.
7. Cold index is ~22s, and the cache only hits with **both** `--head` and `--cache-dir`.
8. Symbols at the repository root report module `""`, so a root-level fixture prints "across 0 modules". Cosmetic, pre-existing, untouched here because changing the module grouping would change `BREAKING`.